Matrices en Python con NumPy

https://gemini.google.com/app/5bcac39086de6681

User prompt: dame esas dos matríces en código para Python con numpy

Response: Aquí tienes el código en Python utilizando `numpy` para definir la matriz A y el vector $\mathbf{b}$ de la imagen:

```
import numpy as np

# Definición de la matriz A (6x6)[cite: 1]
A = np.array([
    [ 3, -1,  0,  0,  0,  0],
    [-1,  3, -1,  0,  0,  0],
    [ 0, -1,  3, -1,  0,  0],
    [ 0,  0, -1,  3, -1,  0],
    [ 0,  0,  0, -1,  3, -1],
    [ 0,  0,  0,  0, -1,  3]
], dtype=float)

# Definición del vector b (como vector unidimensional o vector columna)[cite: 1]
b = np.array([1, 2, 3, 4, 5, 6], dtype=float)

# Si necesitas que 'b' sea estrictamente un vector columna (6x1):
# b_columna = b.reshape(-1, 1)
```

User prompt: esto código para graficar esta bien? si no es asi dime que debo cambiar. # Grafica k_jacobi = np.arange(len(residuos_jacobi)) k_gs = np.arange(len(residuos_gs)) plt.figure(figsize=(15, 10)) plt.semilogy(k_jacobi,residuos_jacobi,"o-", label="Jacobi") plt.semilogy(k_gs,residuos_gs,"s-",label="Gauss-Seidel") plt.axhline(y=tolerancia,color="red",linestyle="--",label="Tolerancia = 1e-8") plt.title("Convergencia de Jacobi y Gauss-Seidel") plt.xlabel("Número de iteraciones k") plt.ylabel("Norma del residuo") plt.legend() plt.grid(True) plt.tight_layout() plt.savefig("residuos.png", dpi=300) plt.show()

Response: Tu código **está muy bien estructurado y es correcto** para graficar la convergencia de los métodos iterativos.

Aquí te detallo por qué funciona perfectamente y un par de detalles menores a tener en cuenta:

Lo que está excelente:

- **`plt.semilogy`**: Es la función ideal para representar la norma del residuo, ya que este suele disminuir de forma exponencial o en varios órdenes de magnitud, facilitando su visualización en escala logarítmica.
- **Eje k**: Usar `np.arange(len(...))` asegura que el número de iteraciones coincida exactamente con la longitud de tus listas o vectores de residuos (empezando desde la iteración 0 o 1 según cómo hayas guardado tu vector inicial).
- **Línea de tolerancia**: `plt.axhline` es muy útil para visualizar claramente el criterio de parada (`tolerancia = 1e-8`).

Pequeñas recomendaciones opcionales:

1. **Tamaño de la figura**: `figsize=(15, 10)` es bastante grande (equivalente a una hoja grande). Si solo quieres mostrarlo en la pantalla o en un reporte estándar de Jupyter/Python, un tamaño como `(8, 5)` o `(10, 6)` suele verse más proporcionado.

2. **Marcadores (`o-`, `s-`)**: Si el número de iteraciones es muy alto (por ejemplo, cientos o miles), los marcadores de puntos y cuadrados (`o` y `s`) van a sobrecargar la línea y hacerla difícil de leer. Si tienes muchas iteraciones, puedes quitarlos y dejar solo las líneas (ej. `plt.semilogy(k_jacobi, residuos_jacobi, label="Jacobi")`).

¡Tu script cumple perfectamente con el objetivo! Puedes ejecutarlo tal cual lo tienes.
